

REDES DE COMPUTADORES

Aula 10 - Camada de Transporte - Parte II

- Protocolos da camada de transporte

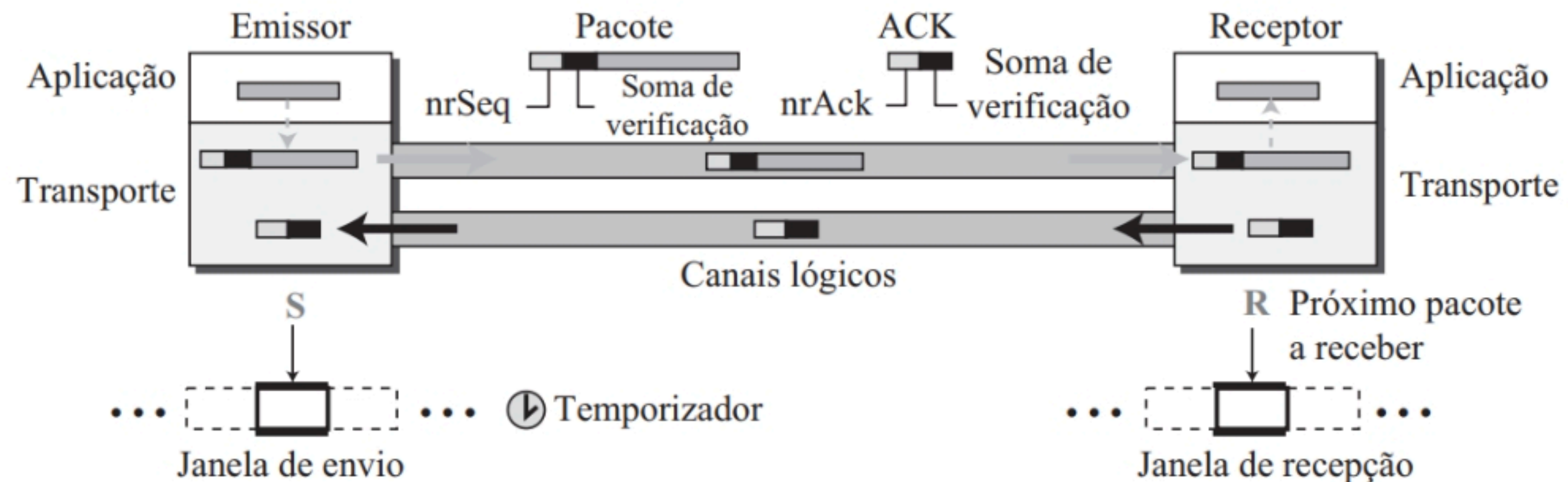


Lorena de S. B. Borges

PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO STOP-AND-WAIT

- PROTOCOLO ORIENTADO À CONEXÃO
- O EMISSOR E O RECEPTOR USAM AMBOS UMA **JANELA DESLIZANTE DE TAMANHO 1**.
- TODA VEZ QUE O EMISSOR ENVIA UM PACOTE, ELE INICIA UM **TEMPORIZADOR**.
- O EMISSOR ENVIA UM PACOTE POR VEZ E ESPERA POR UMA CONFIRMAÇÃO (**ACK**) ANTES DE ENVIAR O PRÓXIMO.
- PARA DETECTAR PACOTES CORROMPIDOS, PRECISAMOS ADICIONAR UMA SOMA DE VERIFICAÇÃO (**CHECKSUM**) A CADA PACOTE DE DADOS.



PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO STOP-AND-WAIT - NÚMEROS DE SEQUÊNCIA

- PARA EVITAR PACOTES DUPLICADOS, O PROTOCOLO USA NÚMEROS DE SEQUÊNCIA E DE CONFIRMAÇÃO (CAMPO ADICIONADO AO CABEÇALHO)
 - TODA VEZ QUE O EMISSOR ENVIA UM PACOTE, ELE INICIA UM TEMPORIZADOR.
1. O pacote chega são e salvo no lado do receptor; este, então, envia uma confirmação, que chega no lado do emissor fazendo com que ele envie o próximo pacote numerado com $x + 1$.
 2. O pacote foi corrompido ou nunca chegou ao receptor; o emissor reenvia o pacote (com a numeração x) após o tempo-limite ser atingido. O receptor retorna uma confirmação.
 3. O pacote chega são e salvo no receptor; este, então, envia uma confirmação, mas ela é corrompida ou perdida. O emissor reenvia o pacote (com a numeração x) após o tempo-limite ser atingido. Perceba que esse pacote foi duplicado. O receptor é capaz de reconhecer esse fato, pois ele espera pelo pacote $x + 1$, mas o pacote recebido é o x .



PROTOCOLOS DA CAMADA DE TRANSPORTE

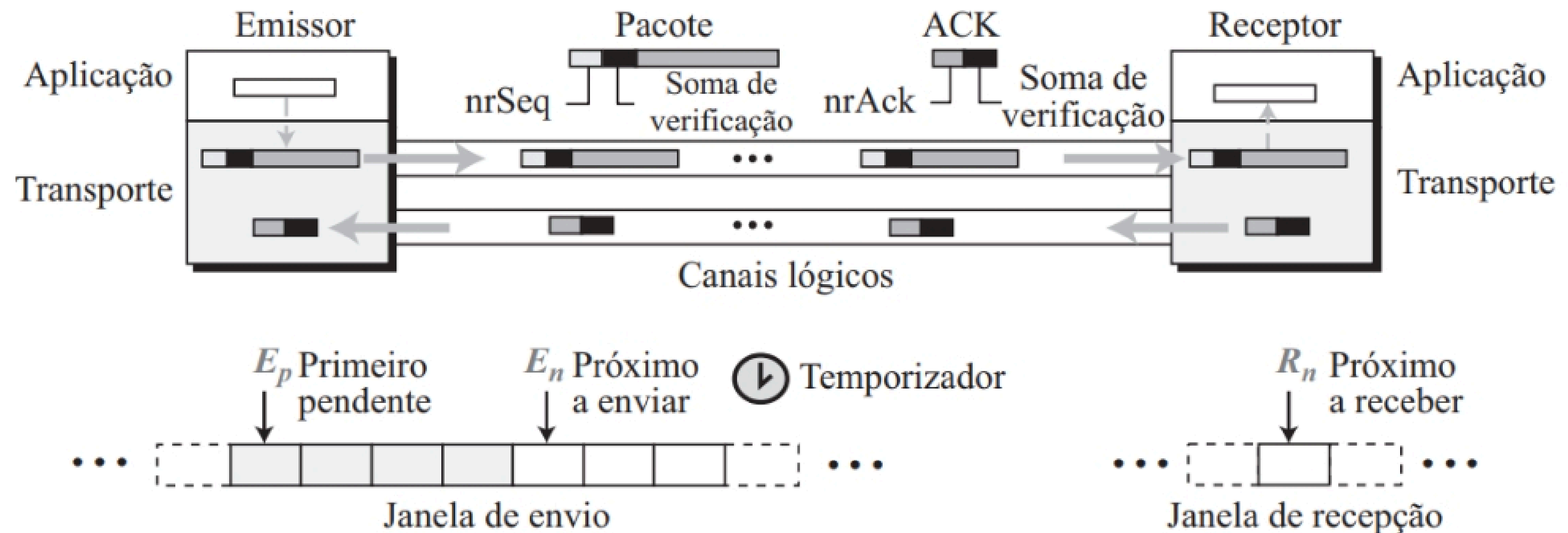
PROTOCOLO STOP-AND-WAIT - NÚMEROS DE CONFIRMAÇÃO

- OS NÚMEROS DE CONFIRMAÇÃO **SEMPRE ANUNCIAM O NÚMERO DE SEQUÊNCIA DO PRÓXIMO PACOTE ESPERADO PELO RECEPTOR.**
- NO PROTOCOLO STOP-AND-WAIT, O NÚMERO DE CONFIRMAÇÃO SEMPRE ANUNCIA, EM ARITMÉTICA MÓDULO 2, O NÚMERO DE SEQUÊNCIA DO PRÓXIMO PACOTE ESPERADO.
- POR EXEMPLO, SE O PACOTE 0 CHEGOU SÃO E SALVO, O RECEPTOR ENVIA UM ACK COM O NÚMERO DE SEQUÊNCIA IGUAL A 1 (OU SEJA, O PACOTE 1 É O PRÓXIMO PACOTE ESPERADO).
- SE O PACOTE 1 CHEGOU SÃO E SALVO, O RECEPTOR ENVIA UM ACK COM O NÚMERO DE SEQUÊNCIA IGUAL A 0 (OU SEJA, O PACOTE 0 É O PRÓXIMO PACOTE ESPERADO).

PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO GO-BACK-N

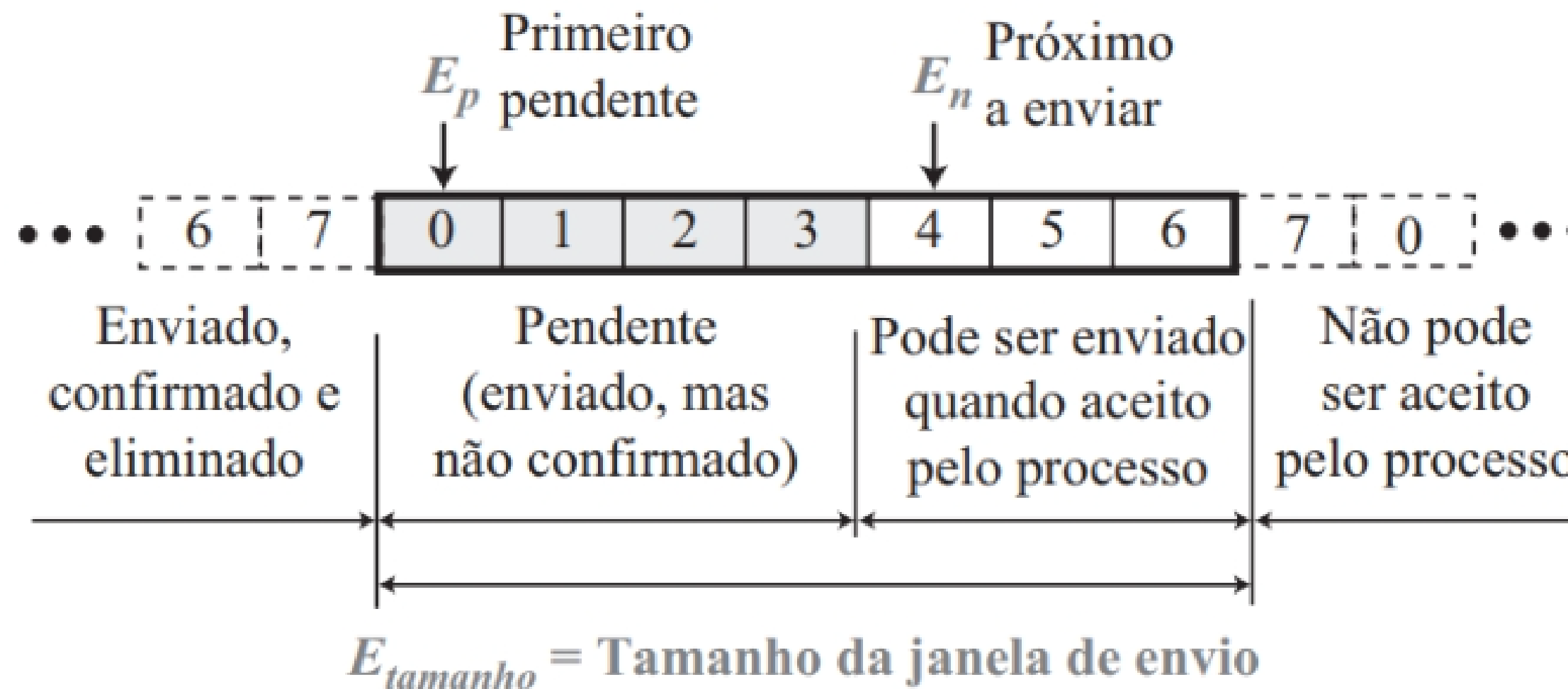
- PARA MELHORAR A EFICIÊNCIA DA TRANSMISSÃO, MÚLTIPLOS PACOTES DEVEM ESTAR EM TRÂNSITO ENQUANTO O EMISSOR ESPERA A CONFIRMAÇÃO
- NO GO-BACK-N PODEMOS ENVIAR DIVERSOS PACOTES ANTES DE RECEBER AS CONFIRMAÇÕES, MAS O RECEPTOR SÓ PODE MANTER UM PACOTE NO BUFFER.



PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO GO-BACK-N

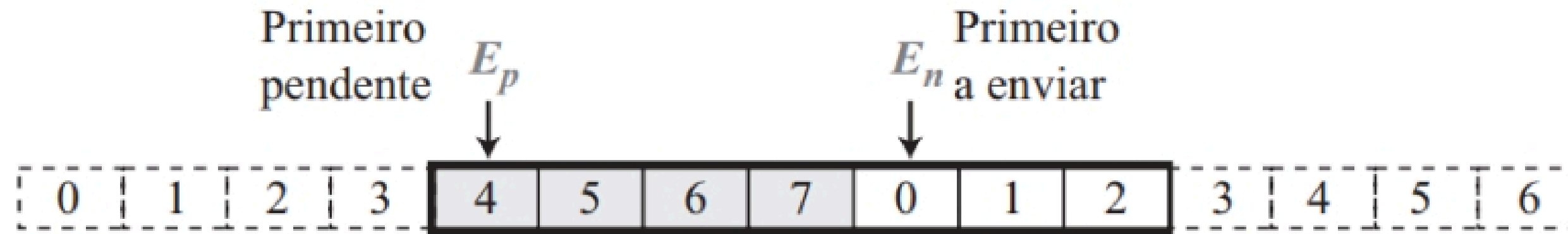
- UM NÚMERO DE CONFIRMAÇÃO NESTE PROTOCOLO É CUMULATIVO E DEFINE O NÚMERO DE SEQUÊNCIA DO PRÓXIMO PACOTE ESPERADO.



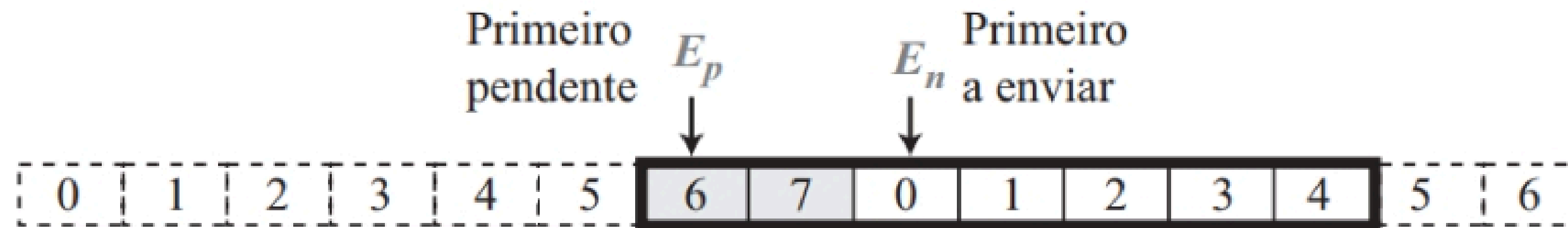
PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO GO-BACK-N

JANELA DESLIZANTE



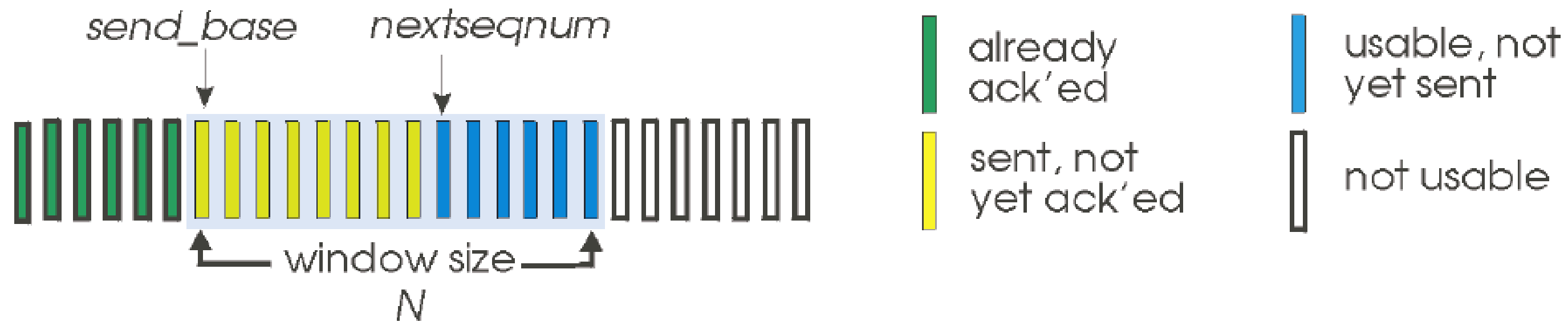
(a) Janela antes do deslizamento



(b) Janela após o deslizamento (um ACK com nrAck = 6 chegou)

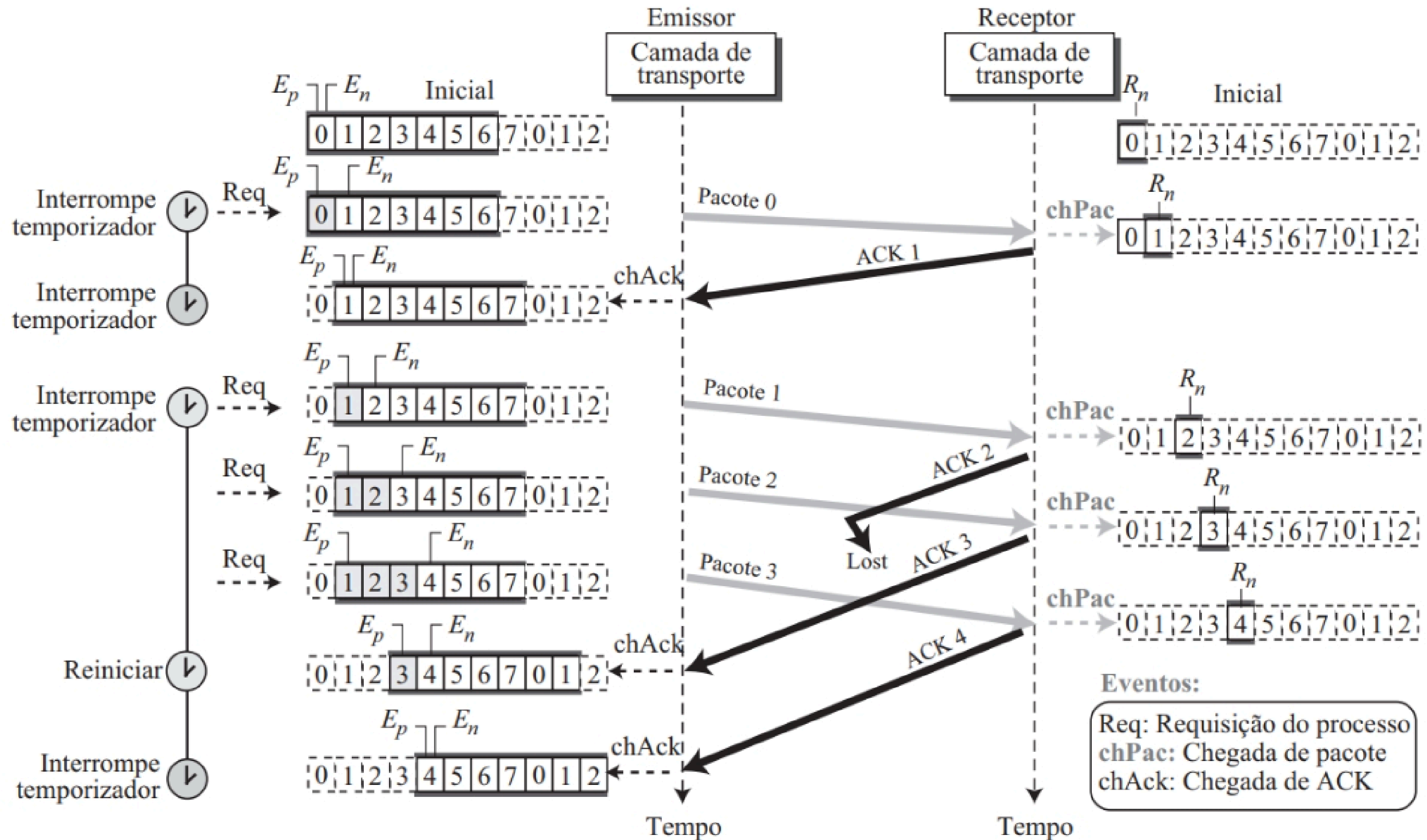
Go-Back-N: sender

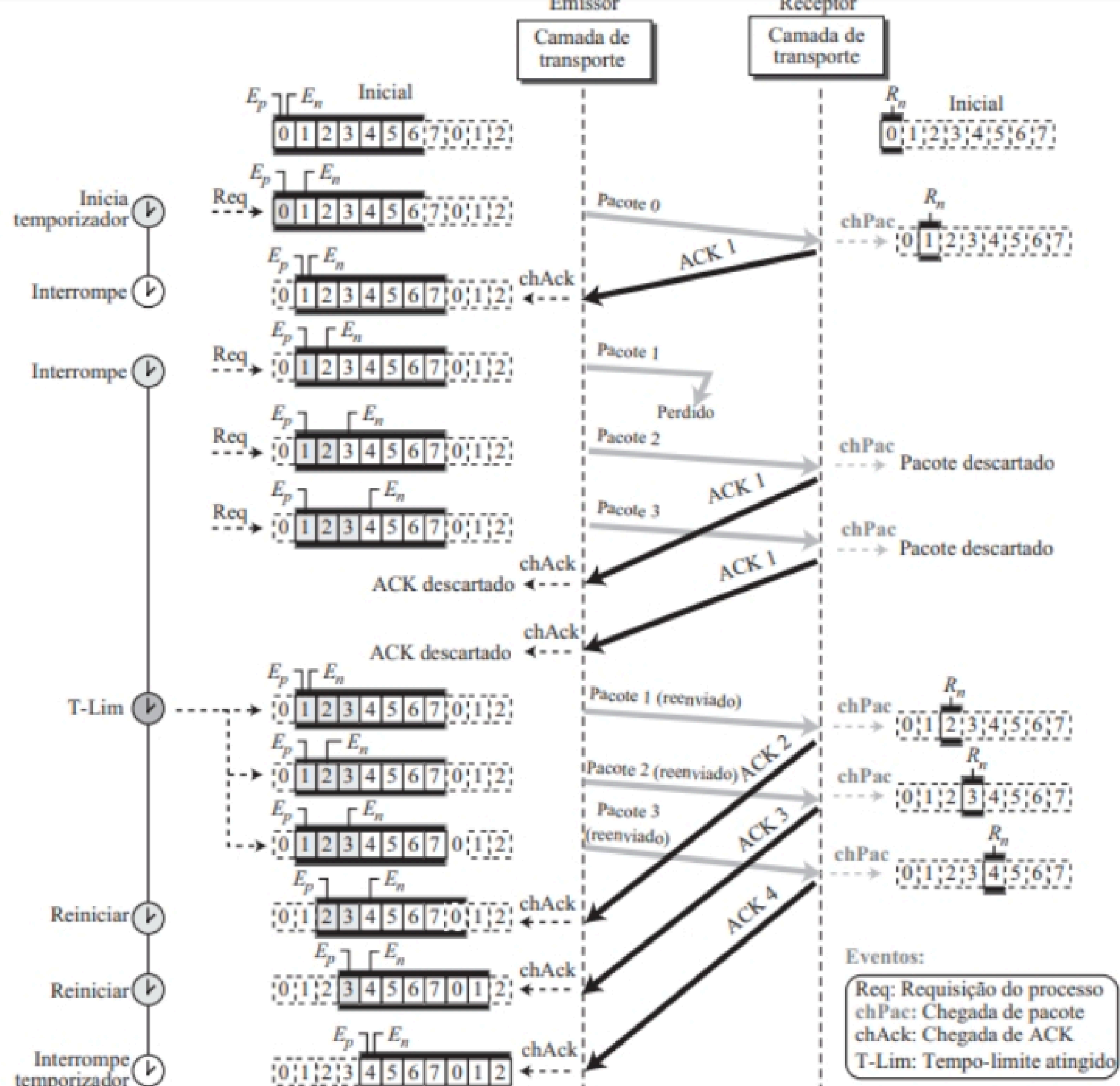
- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - k -bit seq # in pkt header



- *cumulative ACK*: $ACK(n)$: ACKs all packets up to, including seq # n
 - on receiving $ACK(n)$: move window forward to begin at $n+1$
- timer for oldest in-flight packet
- *timeout(n)*: retransmit packet n and all higher seq # packets in window

PROTOCOLOS DA CAMADA DE TRANSPORTE

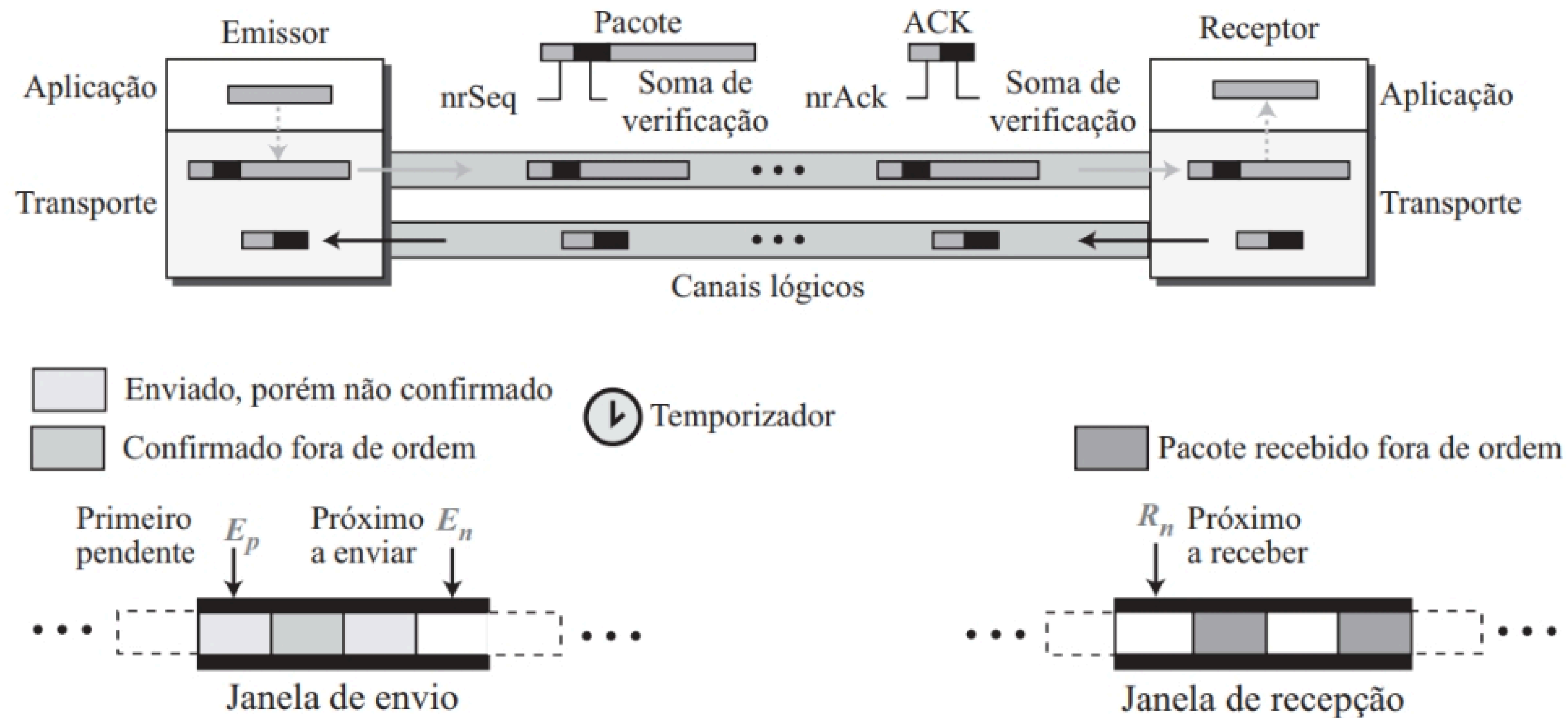




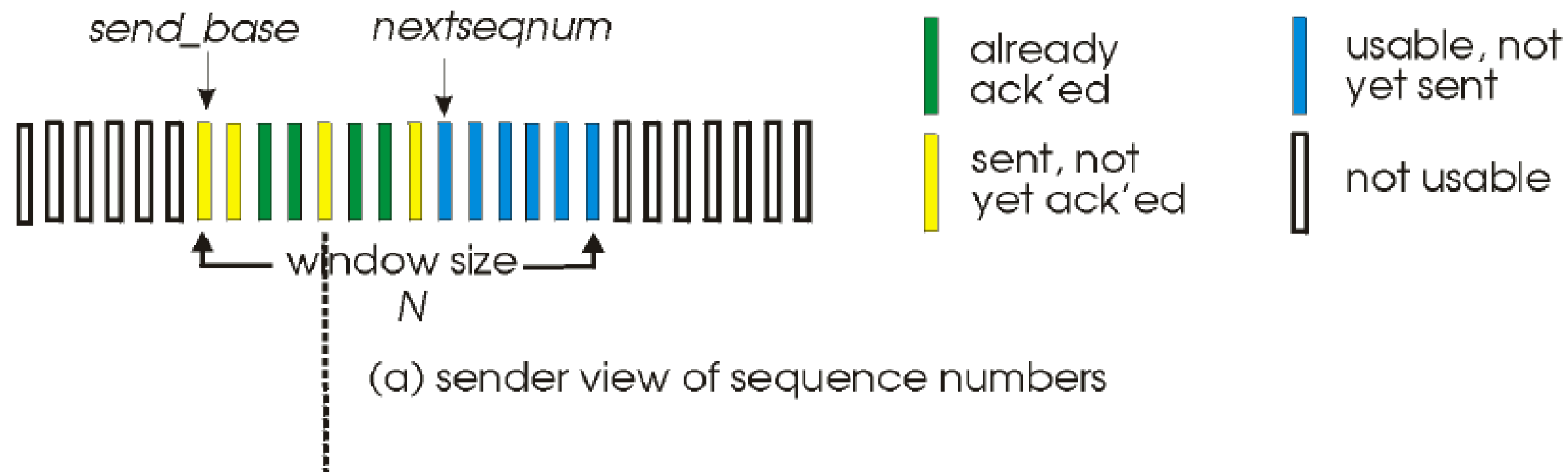
PROTOCOLOS DA CAMADA DE TRANSPORTE

PROTOCOLO REPETIÇÃO SELETIVA (RS)

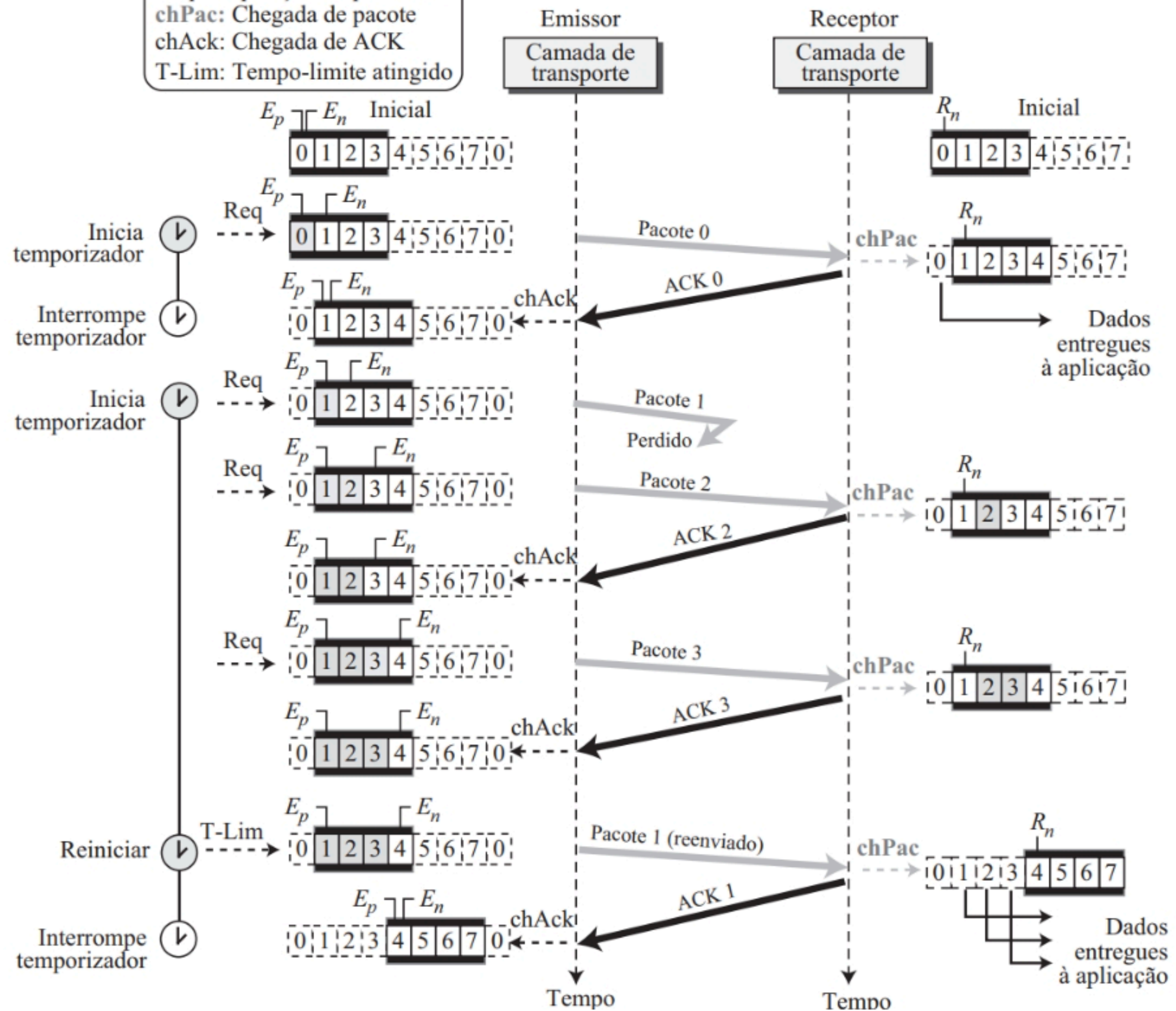
- REENVIAR APENAS PACOTES SELECIONADOS, QUE SÃO AQUELES QUE FORAM REALMENTE PERDIDOS.



Selective repeat: sender, receiver windows



Req: Requisição do processo
 chPac: Chegada de pacote
 chAck: Chegada de ACK
 T-Lim: Tempo-limite atingido





PROTOCOLOS DA CAMADA DE TRANSPORTE

SERVIÇOS DO UDP

- O UDP FORNECE COMUNICAÇÃO PROCESSO A PROCESSO USANDO ENDEREÇOS DE SOCKETS
- O UDP OFERECE UM SERVIÇO **NÃO ORIENTADO À CONEXÃO**
- **CADA DATAGRAMA ENVIADO PELO USUÁRIO É UM DATAGRAMA UDP INDEPENDENTE**
- NÃO FORNECE CONTROLE DE CONGESTIONAMENTO.
- AO CONTRÁRIO DO TCP, **NÃO HÁ ESTABELECIMENTO DE CONEXÃO E NÃO HÁ FINALIZAÇÃO DA CONEXÃO.**
- CADA DATAGRAMA DE USUÁRIO PODE VIAJAR POR UM CAMINHO DIFERENTE.
- NÃO HÁ CONTROLE DE FLUXO E, PORTANTO, NENHUM MECANISMO DE JANELAS.
- NÃO EXISTE UM MECANISMO DE CONTROLE DE ERROS



UDP: User Datagram Protocol

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out-of-order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

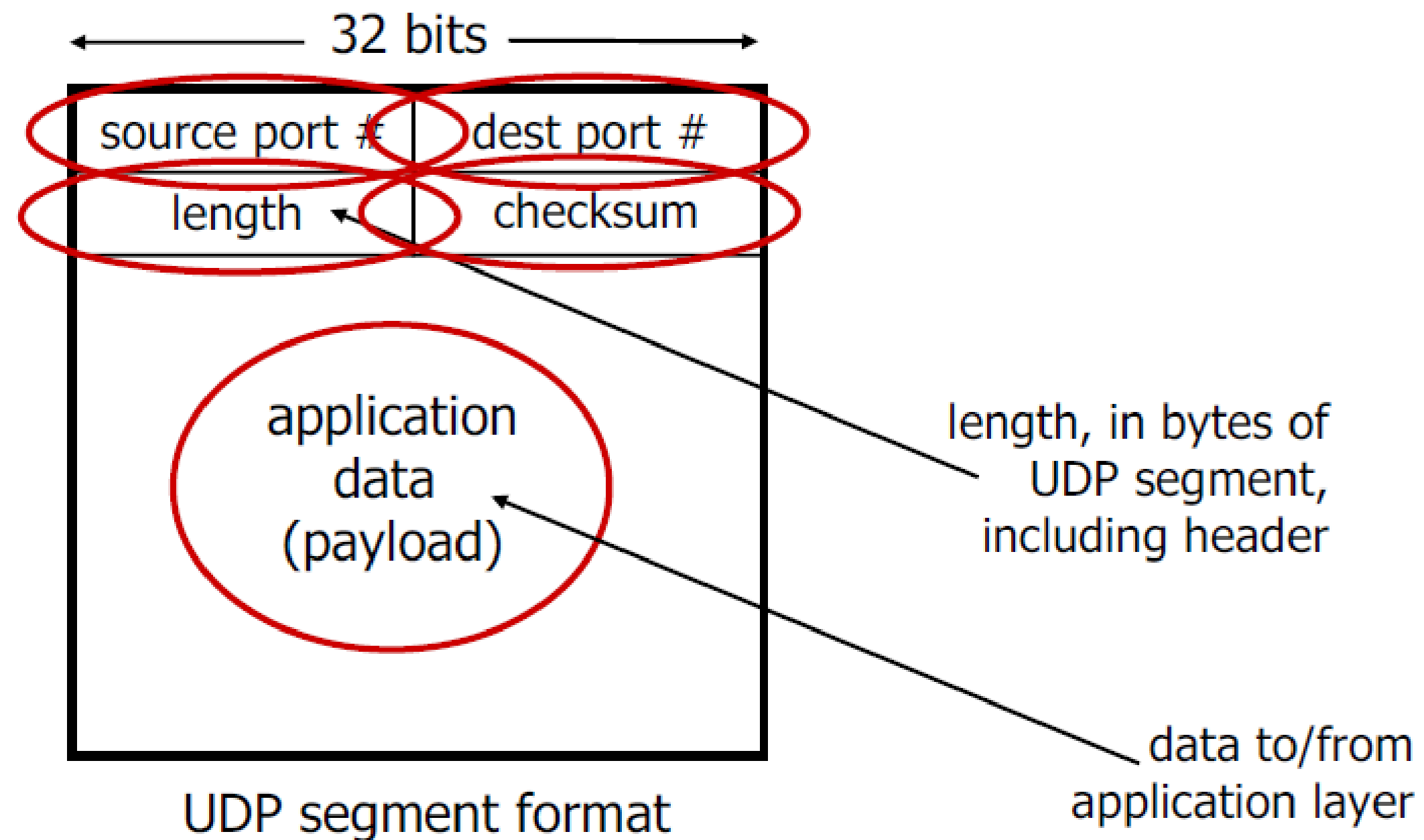
- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
 - UDP can blast away as fast as desired!
 - can function in the face of congestion



UDP: User Datagram Protocol

- UDP use:
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS
 - SNMP
 - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
 - add needed reliability at application layer
 - add congestion control at application layer

UDP segment header



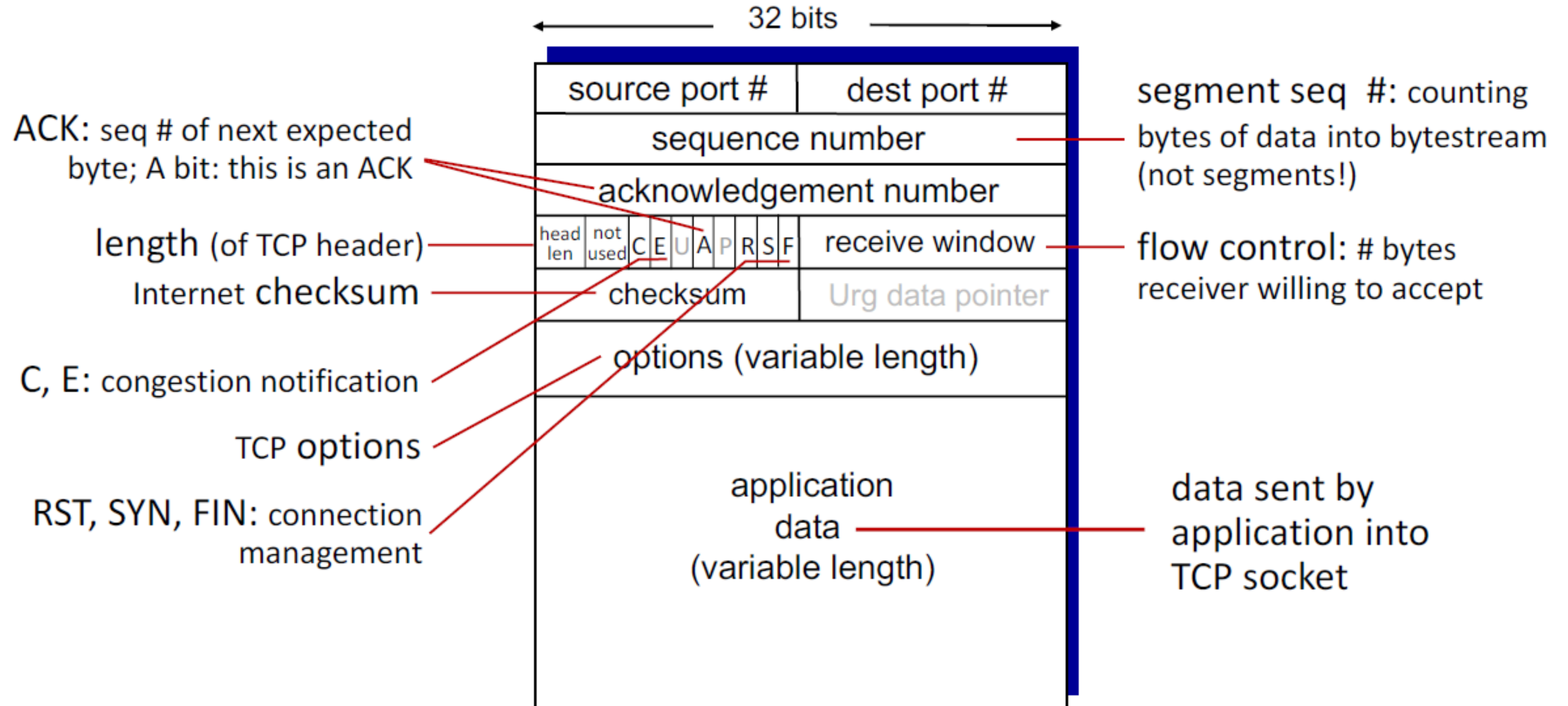


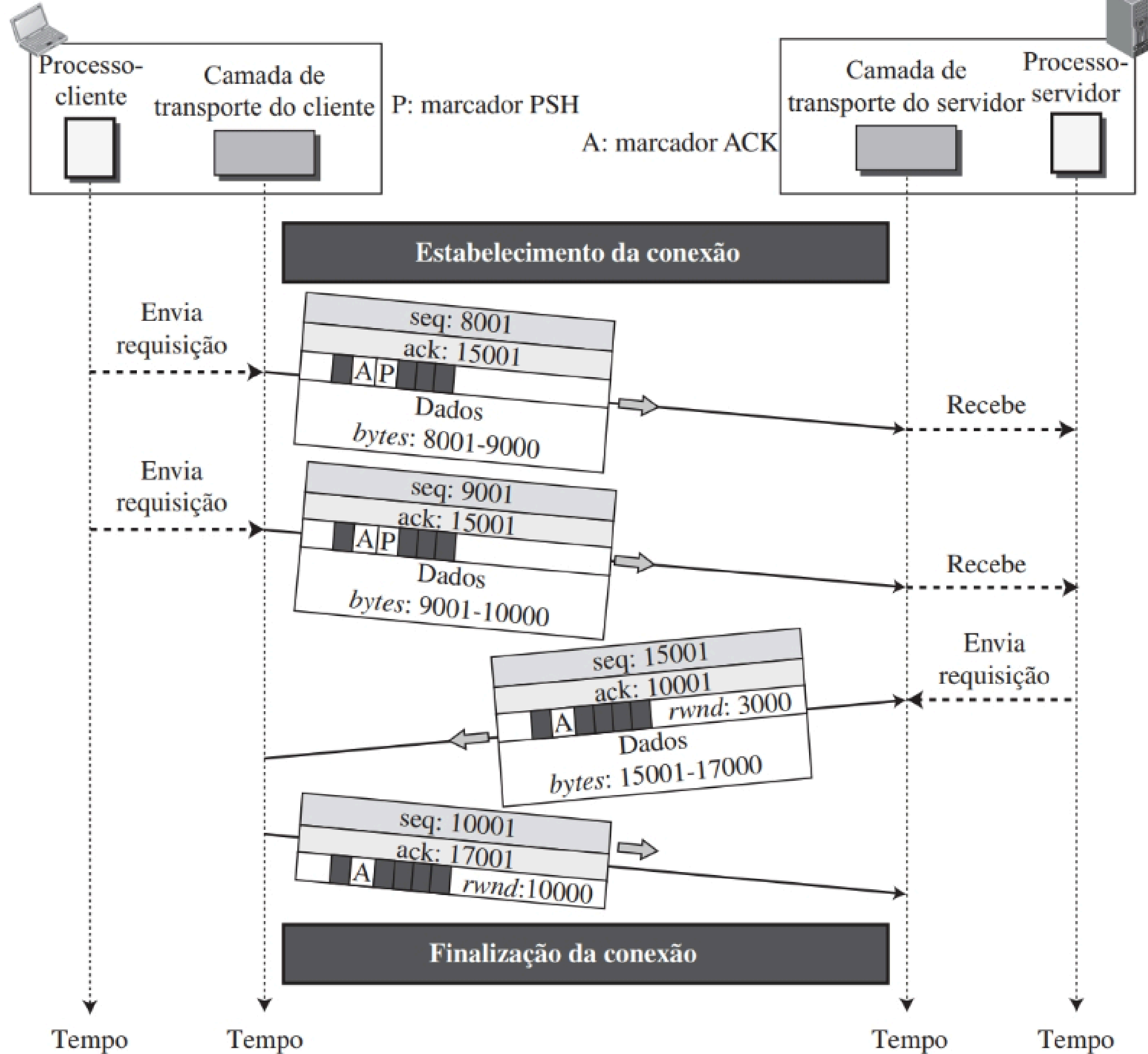
TCP: overview

RFCs: 793, 1122, 2018, 5681, 7323

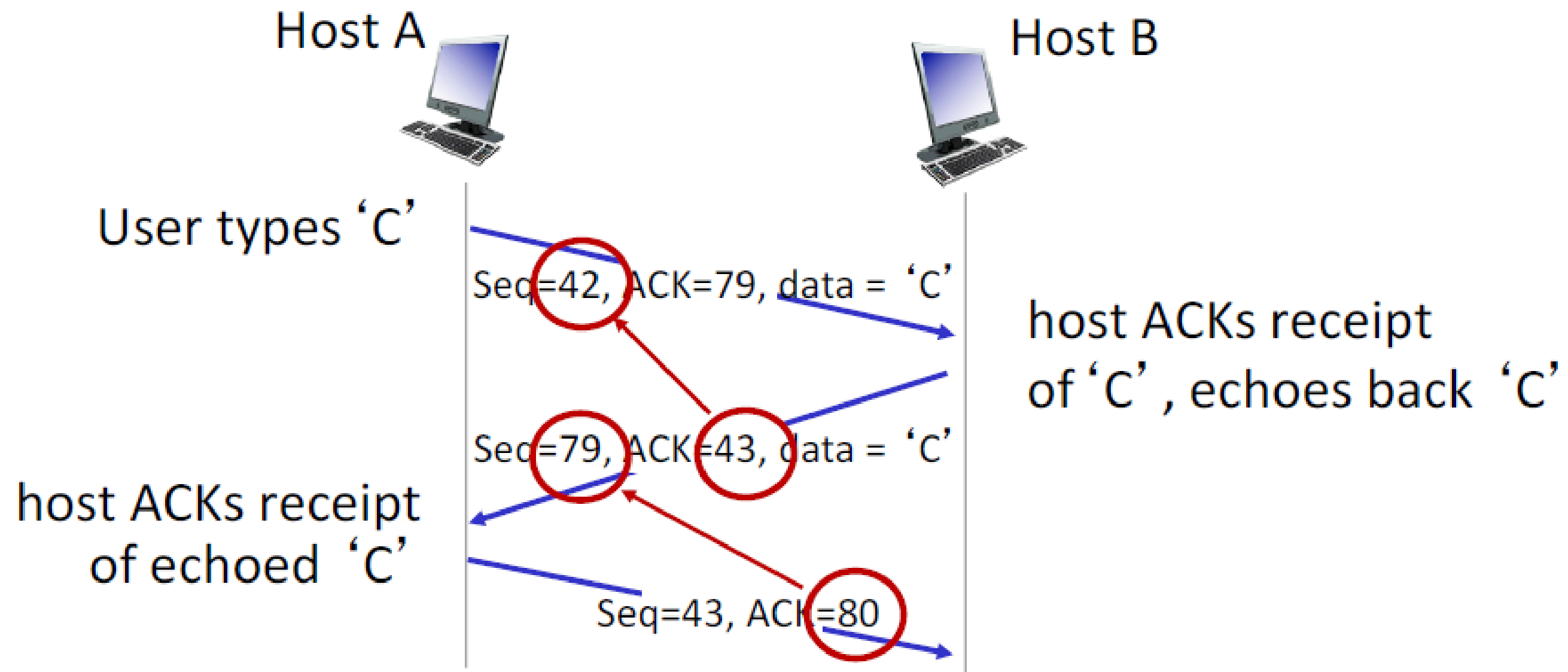
- point-to-point:
 - one sender, one receiver
- reliable, in-order *byte stream*:
 - no “message boundaries”
- full duplex data:
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- cumulative ACKs
- pipelining:
 - TCP congestion and flow control set window size
- connection-oriented:
 - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
 - sender will not overwhelm receiver

TCP segment structure





TCP sequence numbers, ACKs



simple telnet scenario



PROTOCOLOS DA CAMADA DE TRANSPORTE

TCP

- O TCP ATRIBUI UM NÚMERO DE SEQUÊNCIA PARA CADA SEGMENTO QUE ESTÁ SENDO ENVIADO.
- O NÚMERO DE SEQUÊNCIA, EM CADA DIREÇÃO, É DEFINIDO DA SEGUINTE FORMA:
 1. O NÚMERO DE SEQUÊNCIA DO PRIMEIRO SEGMENTO É O NÚMERO DE SEQUÊNCIA INICIAL (ISN – INITIAL SEQUENCE NUMBER), QUE É UM NÚMERO ALEATÓRIO.
 2. O NÚMERO DE SEQUÊNCIA DE QUALQUER OUTRO SEGMENTO É O NÚMERO DE SEQUÊNCIA DO SEGMENTO ANTERIOR MAIS O NÚMERO DE BYTES TRANSPORTADO PELO SEGMENTO ANTERIOR.

PROTOCOLOS DA CAMADA DE TRANSPORTE

TCP

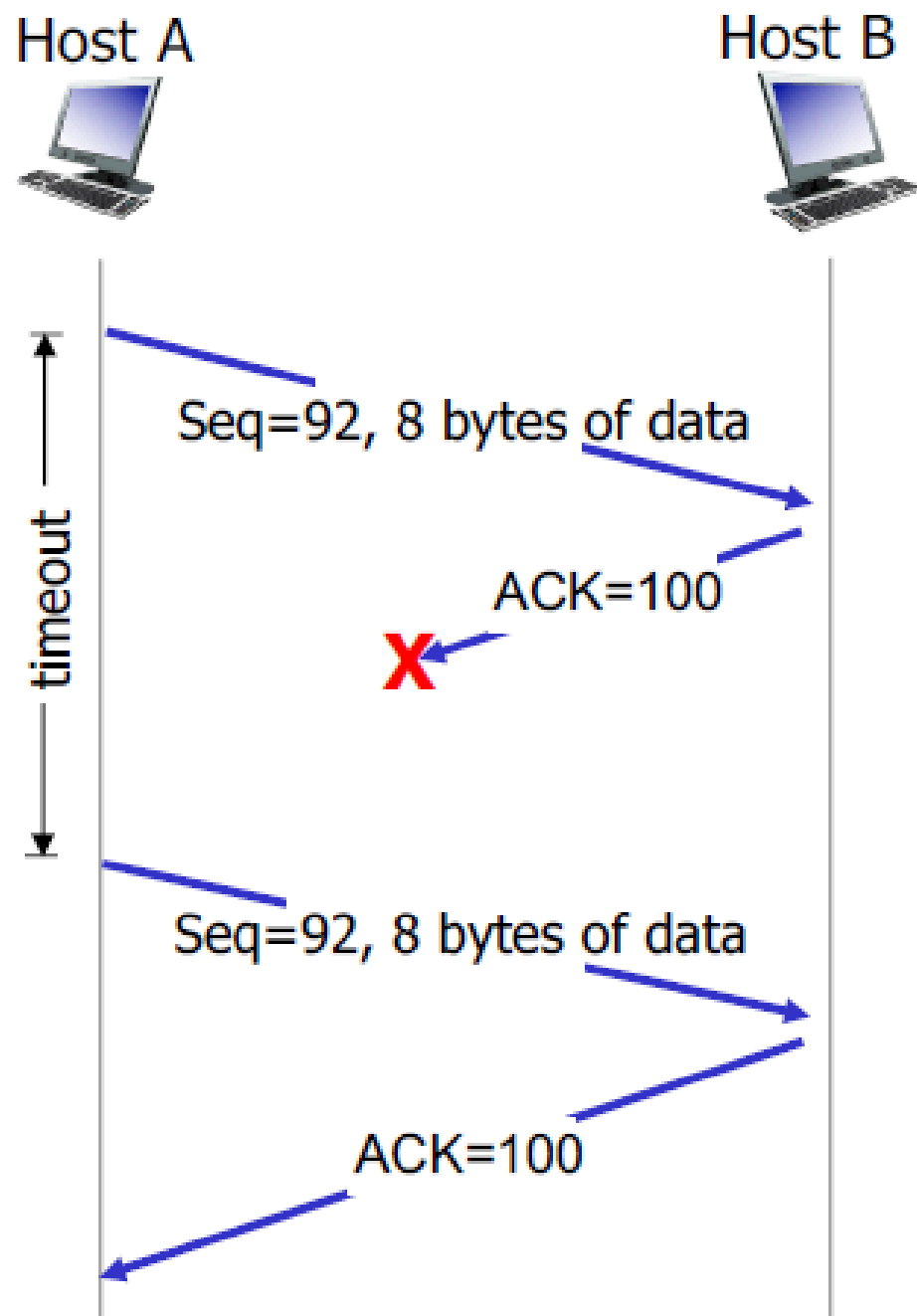
Considere que uma conexão TCP esteja transferindo um arquivo de 5.000 *bytes*. O primeiro é numerado como 10.001. Quais são os números de sequência para cada segmento se os dados forem enviados em cinco segmentos, cada um com 1.000 *bytes*?

Solução

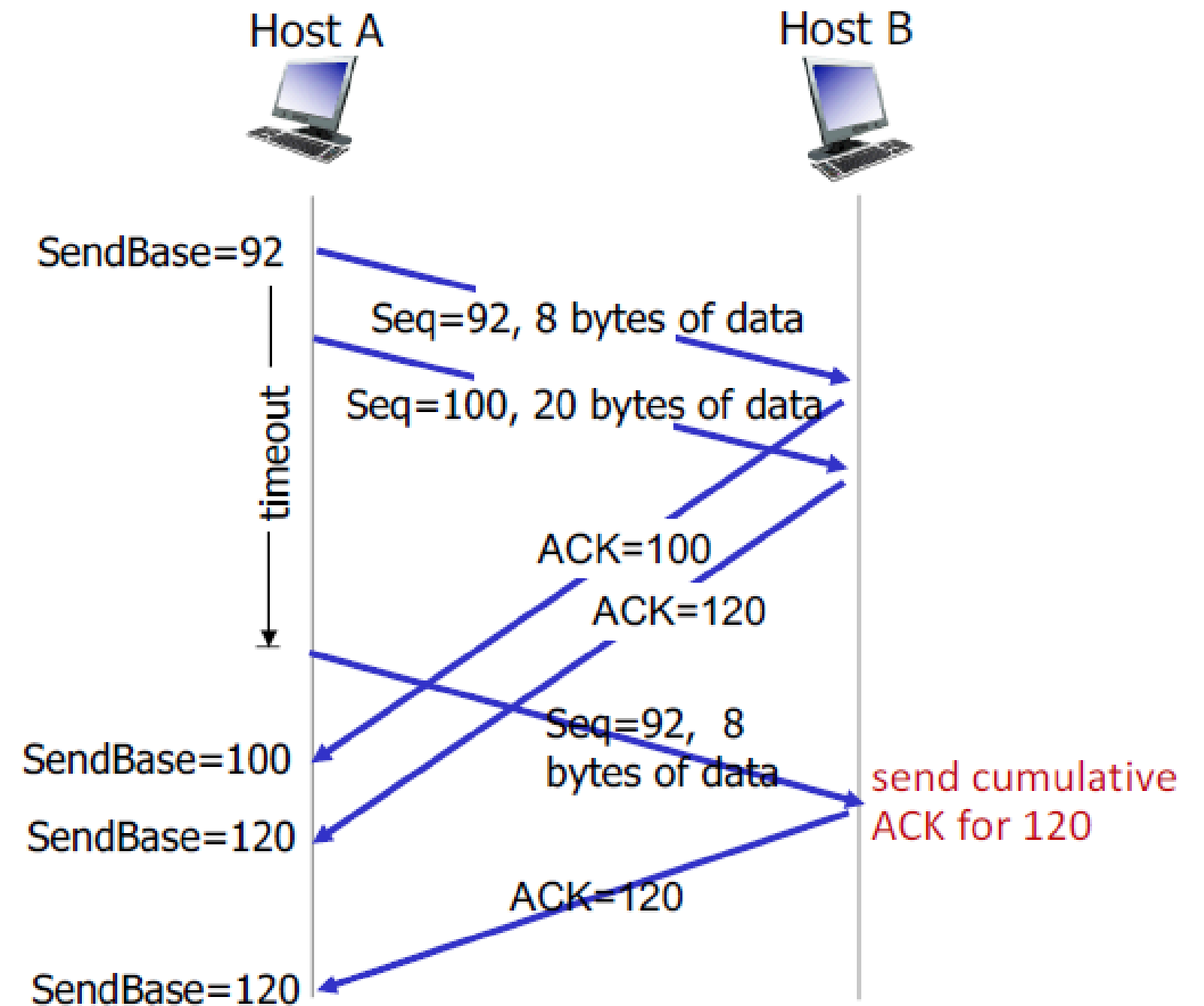
A seguir, são mostrados os números de sequência para cada segmento:

Segmento 1	→	Número de Sequência:	10.001	Faixa:	10.001	a	11.000
Segmento 2	→	Número de Sequência:	11.001	Faixa:	11.001	a	12.000
Segmento 3	→	Número de Sequência:	12.001	Faixa:	12.001	a	13.000
Segmento 4	→	Número de Sequência:	13.001	Faixa:	13.001	a	14.000
Segmento 5	→	Número de Sequência:	14.001	Faixa:	14.001	a	15.000

TCP: retransmission scenarios



lost ACK scenario



premature timeout

A human 3-way handshake protocol



TCP 3-way handshake

Server state

```
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
LISTEN
clientSocket.connect((serverName, serverPort))
```

SYNSENT

ESTAB

choose init seq num, x
send TCP SYN msg

SYNbit=1, Seq=x

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data

ACKbit=1, ACKnum=y+1

received ACK(y)
indicates client is live

LISTEN

SYN RCVD

ESTAB

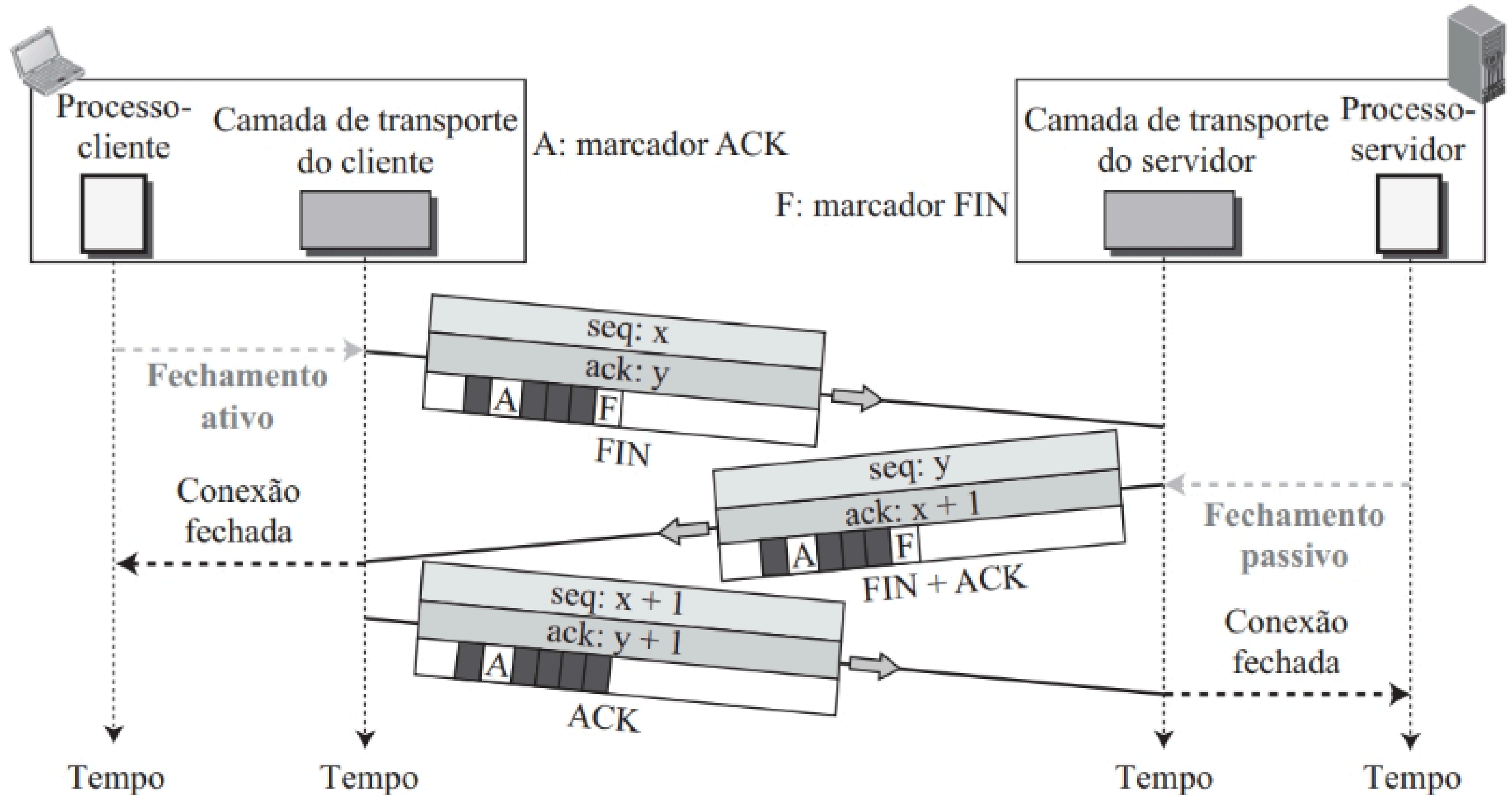




Closing a TCP connection

- client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

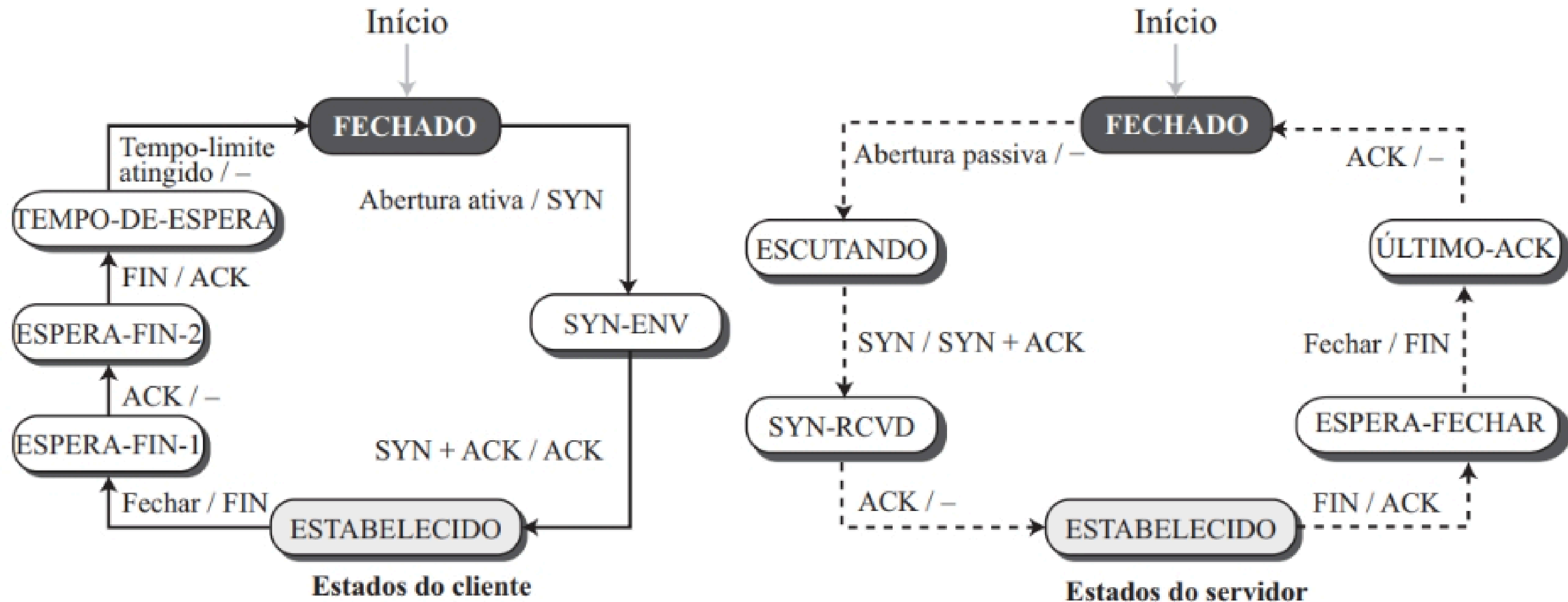
PROTOCOLOS DA CAMADA DE TRANSPORTE

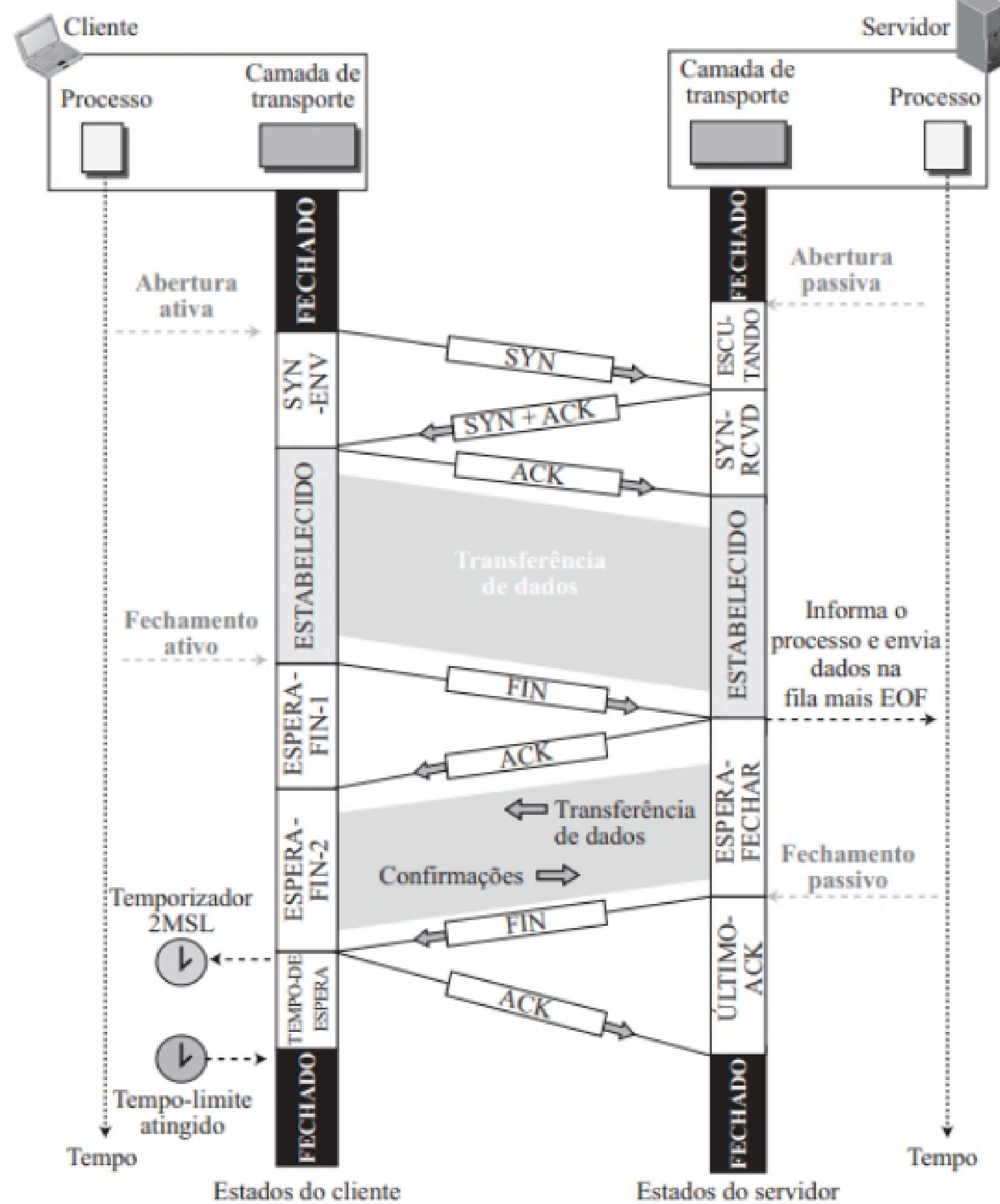


PROTOCOLOS DA CAMADA DE TRANSPORTE

Tabela 3.2 Estados do TCP.	
Estado	Descrição
FECHADO	Não há uma conexão
ESCUTANDO	Abertura passiva recebida; esperando por SYN
SYN-ENV	SYN enviado; esperando por ACK
SYN-RCVD	SYN+ACK enviado; esperando por ACK
ESTABELECIDO	Conexão estabelecida; transferência de dados em andamento
ESPERA-FIN-1	Primeiro FIN enviado; esperando por ACK
ESPERA-FIN-2	ACK para o primeiro FIN recebido; esperando por segundo FIN
ESPERA-FECHAR	Primeiro FIN recebido, ACK enviado; esperando pelo fechamento da aplicação
TEMPO-DE-ESPERA	Segundo FIN recebido, ACK enviado; esperando pela expiração do temporizador 2MSL
ÚLTIMO-ACK	Segundo FIN enviado; esperando por ACK
FECHANDO	Ambos os lados decidem fechar simultaneamente

PROTOCOLOS DA CAMADA DE TRANSPORTE







REFERÊNCIAS:

- [1] Capítulo 3 - FOROUZAN, B. A. Redes de Computadores: Uma Abordagem Top-Down. AMGH, 2013.
- [2] Capítulo 3 - Computer Networking: A Top-Down Approach 8th edition Jim Kurose, Keith Ross Pearson, 2020.

REDES DE COMPUTADORES

Aula 10 - Camada de Transporte - Parte II

- Protocolos da camada de transporte



Lorena de S. B. Borges